

FinSight India — 6-Week Agentic AI + RAG Plan (Cash Equity + F&O)

An agentic research assistant over NSE/BSE filings, earnings concalls, derivatives data, and SEBI regulations — covering **both cash equities and F&O from week 1**, restructured around the order from the post you shared: **tool calling → agent loop → RAG → memory → evals → guardrails + deploy**.

This order is right for your background. Building the agent loop *before* RAG forces you to understand orchestration on its own terms, instead of RAG and agents getting tangled together from day one. By week 3, retrieval slots in as just another tool the agent already knows how to call.

Stack (all free): LLM: Google Gemini API (primary) + Groq (fast/backup) · Orchestration: LangGraph · Embeddings: sentence-transformers (local) · Vector DB: Chroma (local) · Reranker: local cross-encoder · Web search: Tavily free tier · Cash equity data: yfinance with [.NS/.BO](#) tickers · F&O + macro data: [jugaad-data](#) or [nsepython](#) (option chains, OI, bhavcopy, RBI repo rate) · Filings/concalls: NSE/BSE corporate announcements pages + company IR pages · Regulations: SEBI circulars (sebi.gov.in, free/public) · Eval: RAGAS · Deploy: Streamlit Community Cloud / HuggingFace Spaces

Two things that touch every week: report figures in ₹ crore/lakh, not billions, since that's what you'll be feeding the model and what it should echo back. And NSE's site has anti-scraping protection ([jugaad-data/nsepython](#) build in caching/throttling to handle this) — the retry/backoff habit matters for market-data tools, not just LLM rate limits.

Rhythm for 15+ hrs/week: roughly 2.5–3 hrs/day, 6 days/week. Treat one day as a "break it and fix it" day — debugging is most of the actual learning here.

Week 1 — Call an LLM API + Tool Calling

Goal: Get comfortable with the raw mechanics of function/tool calling, across both cash equity and F&O data sources, before any agent abstraction touches it.

Concepts required to know: message format and roles; tool/function schemas (JSON schema for parameters); how the model "decides" to call a tool vs. respond directly; parsing tool-call responses and feeding results back; basic F&O vocabulary you'll need for the tools below (option chain, open interest, strike, expiry, lot size).

Practical tasks / milestones: A single Python script, no framework. Build 5 tools:

- `get_stock_price(ticker)` — yfinance, `.NS` tickers (e.g. `TCS.NS`)
- `get_company_overview(ticker)` — fundamentals via yfinance
- `get_option_chain(symbol)` — via `jugaad-data/nsepython`, for index or stock F&O (e.g. NIFTY weekly chain)
- `get_repo_rate()` — current RBI repo rate via `jugaad-data`'s RBI module
- `calculate(expression)` — simple safe calculator

Manually wire the round trip for each: model proposes a call → you execute the function → result goes back to the model → model answers. Do this with **both** Gemini and Groq so you see tool-calling is provider-agnostic, just with slightly different JSON shapes.

Day to day:

- Day 1–2: Get both API keys, make a first plain-text call to each, understand request/response shape.
- Day 3–4: Read each provider's function-calling docs, define schemas for the 3 cash-equity/utility tools, get the model to *propose* a call (don't execute yet).
- Day 5: Wire up execution + round-trip for those 3 tools.
- Day 6: Add `get_option_chain` and `get_repo_rate` the same way — same pattern, new data source.
- Day 7: Full end-to-end test of all 5 tools with both Gemini and Groq.

References: Gemini/Groq function-calling docs, `jugaad-data/nsepython` docs for the option-chain and RBI endpoints, Anthropic's "Building Effective Agents" post (provider-agnostic framing, worth reading now).

Main output of the week: A working script with 5 tools spanning cash equities, F&O, and macro context, each independently tested end-to-end with two different LLM providers.

Week 2 — Add a Loop = Your First Agent

Goal: Turn the one-shot tool calls into a loop that chains multiple tools across *both* domains and reasons between them, without you scripting the order.

Concepts required to know: the ReAct pattern (Reason → Act → Observe, repeat); stopping conditions (max iterations, "final answer" detection); handling a tool that fails or returns nothing; state management across loop iterations.

Practical tasks / milestones: Introduce LangGraph — explicit state graphs save real pain once memory, RAG, and guardrails are all touching the same loop later. Port week 1's 5 tools in as LangGraph tool nodes. Target query: *"Is TCS's P/E higher than Infosys's, and is there unusual options activity building up in NIFTY this week?"* — this needs 4+ tool calls across cash and F&O data, with the model deciding the order itself.

Day to day:

- Day 1–2: LangGraph quickstart — nodes, edges, state.
- Day 3–4: Port all 5 tools in, build the loop with a max-iteration safety cap.
- Day 5: Deliberately break it — ask something needing a tool you haven't built, confirm it fails gracefully instead of hallucinating.
- Day 6–7: Add step-by-step logging (tool, args, result) — you'll reuse this in week 5 for evals.

References: LangGraph docs/tutorials, the ReAct paper (skim — the core idea is short).

Main output of the week: A LangGraph agent that autonomously chains tool calls across cash equity and F&O data sources to answer a multi-part question, with logging and a safety cap on loop length.

Week 3 — Add RAG

Goal: Ground answers in real documents — company filings/concalls *and* SEBI regulatory text — instead of only live numbers.

Concepts required to know: chunking strategy for messy financial documents (tables, scanned PDFs, footnotes); embeddings; vector similarity search; hybrid retrieval (keyword + embedding); reranking; basic metadata filtering (by doc type, company, date).

Practical tasks / milestones:

1. Pull quarterly results PDFs and concall transcripts for 5–10 NSE-listed companies (e.g. TCS, Infosys, Reliance, HDFC Bank, ICICI Bank, Tata Motors) from NSE/BSE corporate announcements or company IR pages.
2. Pull SEBI circulars relevant to F&O regulation (margin rules, lot-size changes, expiry-day rules) from sebi.gov.in.
3. Chunk and embed both sets locally (sentence-transformers), store in Chroma with a `doc_type` metadata field (`filing` vs `regulation`).
4. Add hybrid retrieval + a local cross-encoder reranker.
5. Wrap as two tools: `search_filings(query, ticker, fiscal_period)` and `search_regulations(query)` — give both to the week 2 agent.

Target query: *"What did Reliance's Q3 concall say about Jio's subscriber growth, and has SEBI changed F&O lot-size rules recently?"* — needs both tools, and the agent should pick correctly on its own.

Day to day:

- Day 1–2: Download and chunk company filings/concalls (scanned PDFs and inconsistent layouts will eat time here — budget for it).
- Day 3: Download and chunk SEBI circulars (cleaner text, faster to process).
- Day 4: Embed + load both sets into Chroma with metadata tagging.
- Day 5: Basic retrieval, sanity-check results manually across both doc types.
- Day 6: Add hybrid search + reranking.
- Day 7: Wire both search tools into the week 2 agent, test multi-hop queries spanning filings, regulations, and live data.

References: A RAG-from-scratch tutorial (LangChain or LlamaIndex), NSE/BSE corporate filings pages, sebi.gov.in circulars.

Main output of the week: An agent that retrieves and cites from two distinct grounded sources — company filings/concalls and SEBI regulatory text — choosing the right one per query, on top of the existing cash/F&O tool layer.

Week 4 — Add Memory

Goal: Stop the agent from forgetting what it's tracking — across both equity positions and F&O instruments — between turns.

Concepts required to know: short-term memory (conversation buffer within a session); longer-term/session state (a persisted watchlist); context window management (summarizing old turns instead of keeping everything verbatim).

Practical tasks / milestones: Multi-turn handling where the watchlist can mix instrument types:

- User: *"Track HDFC Bank, ICICI Bank, and NIFTY weekly options."*
- User (later): *"How are their margins and OI trends looking?"* → agent resolves "their" from memory and correctly applies margin to the banks and OI to the NIFTY options, not the other way around.

Persist the watchlist to SQLite or JSON so it survives between sessions. Add summarization once conversations get long enough to risk context limits — a good point to also stress-test your retry/backoff logic under realistic multi-turn load.

Day to day:

- Day 1–2: Add conversation buffer memory in LangGraph.
- Day 3–4: Add persisted, mixed-instrument-type watchlist state with file storage.
- Day 5: Add summarization for long conversations.
- Day 6–7: Stress-test a 15+ turn conversation mixing equity and F&O references, confirm correct resolution and that rate-limit handling holds.

References: LangGraph's memory/persistence docs.

Main output of the week: A multi-turn session that maintains a persisted, mixed cash-equity-and-F&O watchlist with correct pronoun/reference resolution across turns.

Week 5 — Add Evals

Goal: Prove the agent works across all its domains — cash, F&O, filings, and regulations — instead of relying on "it felt right when I tested it."

Concepts required to know: building a held-out eval set; RAG-specific metrics (faithfulness, context precision/recall) via RAGAS; agent-level metrics (task success rate, tool-call correctness, did it pick the right tool/source); latency and cost tracking.

Practical tasks / milestones: Write 25–35 questions spanning all five tool types (live price, fundamentals, filings/concall search, option chain/OI, SEBI regulation search) plus multi-hop combinations, with known correct answers/expected sources. Run RAGAS separately on the filings corpus and the SEBI corpus. Score tool-call correctness from your week 2 logs. Find and fix your worst 3 failure modes — pay particular attention to whether the agent ever confuses a regulation answer with a filing answer, since that's the most India-specific failure mode this build can have.

Day to day:

- Day 1–2: Write the eval set, covering all five tool types and a few deliberately ambiguous multi-hop questions.
- Day 3–4: Run RAGAS on both retrieval corpora, interpret scores.
- Day 5: Score tool-call correctness from logs.
- Day 6–7: Fix the worst failure modes, re-run to confirm improvement.

References: RAGAS documentation.

Main output of the week: A scored eval report covering cash, F&O, filings, and regulatory query types, with the top failure modes identified and at least partially fixed.

Week 6 — Add Guardrails + Deploy

Goal: Make it trustworthy enough to show someone, and put it behind a public URL.

Concepts required to know: citation enforcement (no claim without a source); refusing gracefully when retrieval comes back empty; input validation; cost/rate-limit ceilings so a runaway loop can't drain your free-tier quota; why F&O outputs specifically need a clear "informational, not trading advice" framing given how much retail F&O activity SEBI itself has flagged concerns about.

Practical tasks / milestones:

1. Enforce citations on every filings- and regulation-based answer; if nothing relevant is retrieved, the agent says so instead of guessing.
2. Add an explicit disclaimer whenever the agent surfaces option-chain/OI data, framing it as research output, not a trade recommendation.
3. Add a hard cap on loop iterations and tool calls per query.
4. Build a Streamlit UI with two views: a cash-equity research view and an F&O/OI view.
5. Deploy on Streamlit Community Cloud or HuggingFace Spaces.
6. Write a README with an architecture diagram (tools, agent loop, dual RAG sources, memory layer) — this is what a recruiter or interviewer actually looks at.

Day to day:

- Day 1–2: Citation enforcement + graceful "no answer" handling.
- Day 3: F&O disclaimer logic + iteration/cost caps.
- Day 4–5: Streamlit UI (both views).
- Day 6: Deploy.
- Day 7: README + architecture diagram + final end-to-end test pass.

References: Streamlit deployment docs, HuggingFace Spaces docs.

Main output of the week: A deployed, publicly accessible app handling cash equity and F&O research with enforced citations, a trading-advice disclaimer where relevant, and documentation suitable for a portfolio.

Optional Week 7 — Multi-Agent Stretch

Goal: Add a verification layer without starting a new project.

Concepts required to know: multi-agent coordination patterns, critic/verifier agent design, claim-checking against retrieved sources.

Practical tasks / milestones: Split the single agent into a "research agent" that drafts the answer and a "critic agent" that checks every claim against retrieved filings/regulations/data before the answer ships. Build with LangGraph subgraphs or CrewAI.

Day to day:

- Day 1–3: Design the research/critic split and the handoff contract between them.
- Day 4–5: Implement the critic agent's claim-checking logic.
- Day 6–7: Test on the week 5 eval set, compare accuracy with and without the critic layer.

References: LangGraph multi-agent/subgraph docs, CrewAI docs.

Main output of the week: A two-agent pipeline where a critic verifiably catches at least some unsupported claims the single-agent version missed.

Why this sequencing works for you specifically

Starting with raw tool calling (week 1) before any framework means LangGraph in week 2 reinforces a pattern you already understand, instead of introducing magic. Putting RAG in week 3 — *after* the agent loop exists — means retrieval slots in as "just another tool," exactly how it behaves in a real production agent. Evals before guardrails (week 5 before 6) means you fix problems you can actually measure, instead of guessing at what needs guarding against. And folding F&O and SEBI regulations in from week 1 rather than bolting them on later means your tool-selection logic has to handle ambiguity (cash vs. derivatives vs. regulatory) from the start, which is the harder and more realistic version of the problem.